SPRINT CYCLE II · DATABASE DESIGN

Rattler Resource Rescue System

The data behind sharing textbooks, electronics, furniture and supplies across campus.

Lloyd Lewis · Ondrea Wagner · Naser Halloum · Isaiah Miller

OVERALL DESIGN

One relational database for the complete system

20
entities

127
fields in the data dictionary

34
relationships

4
associative entities

Whole system

Built from the full Functional Requirements and all five roles, not just the Sprint I screens.

MySQL + Django ORM

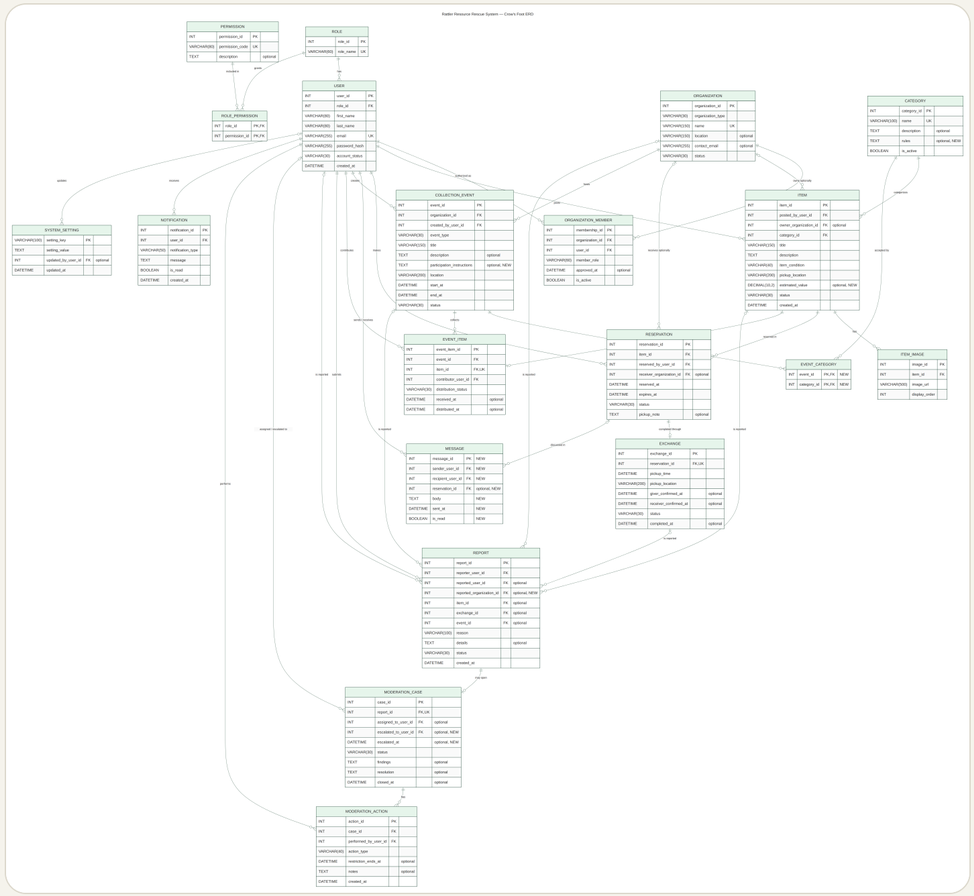
InnoDB tables with foreign keys and transactions, defined as Django models in Sprint III.

Traceable

Listings, reservations, exchanges and moderation actions are all timestamped.

ENTITY-RELATIONSHIP DIAGRAM

The complete ERD



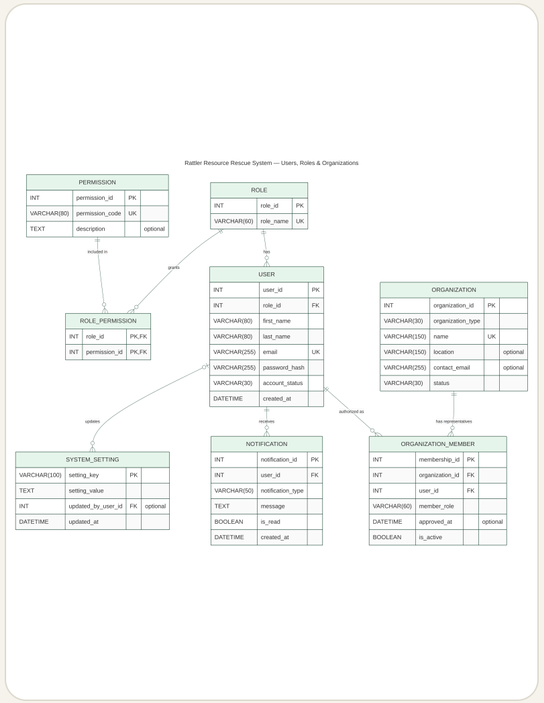
Crow’s foot notation

Symbol	Meaning
`||`	Exactly one
`|o`	Zero or one
`o{`	Zero or many
PK / FK / UK	Primary, foreign, unique key
optional	Field may be empty
NEW	Added during team review

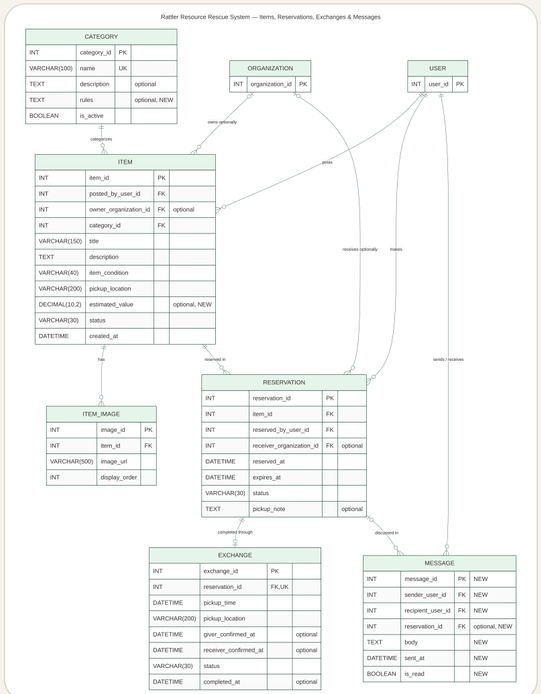
Full-size, zoomable copy: `ERD/Sprint-II-ERD.pdf`

ENTITY-RELATIONSHIP DIAGRAM

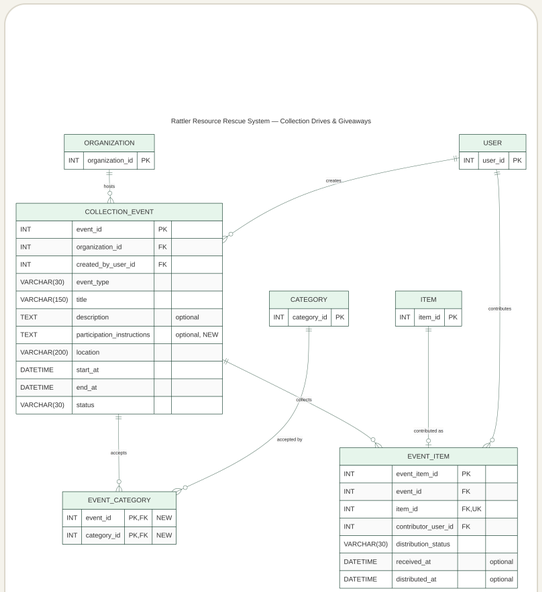
The ERD in four readable areas



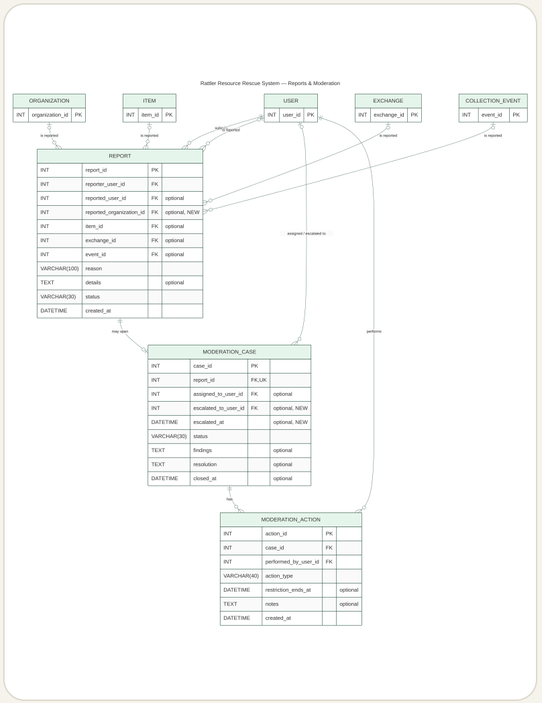
1 · Users, roles & organizations



2 · Items, reservations & exchanges



3 · Collection drives



4 · Reports & moderation

MAJOR ENTITIES

What each major entity represents

Entity	Represents
USER	A person with an account, holding one of the five roles
ORGANIZATION	A department or registered student organization (organization_type)
CATEGORY	A type of item, e.g. Textbooks, Electronics, Furniture
ITEM	A listing offered by a user or an organization, with its photos (ITEM_IMAGE)
RESERVATION	A claim on an item by a user, optionally for an organization
EXCHANGE	The pickup or transfer that completes a reservation
COLLECTION_EVENT	A giveaway or collection drive run by a student organization
REPORT	A problem reported about a listing, user, organization, exchange or event
MODERATION_CASE	A moderator’s case opened from a report, with its actions
NOTIFICATION	A system message to a user about relevant activity

Also: ROLE, PERMISSION, ITEM_IMAGE, MESSAGE, EVENT_ITEM, MODERATION_ACTION, SYSTEM_SETTING

RELATIONSHIPS & CARDINALITY

The relationships that matter most

Parent	Cardinality	Child	Meaning
ROLE	1 : M	USER	Each user has exactly one role
USER	1 : M	ITEM	A user posts many listings
CATEGORY	1 : M	ITEM	Every item belongs to one category
ORGANIZATION	0..1 : M	ITEM	An item may be owned by an organization
ITEM	1 : M	RESERVATION	Many over time, one active at a time
USER	1 : M	RESERVATION	A user makes many reservations
RESERVATION	1 : 0..1	EXCHANGE	One-to-one: unique reservation_id
REPORT	1 : 0..1	MODERATION_CASE	One-to-one: a report opens at most one case
MODERATION_CASE	1 : M	MODERATION_ACTION	A case records many actions

All 34 relationships: Data Dictionary, Relationships tab

MANY-TO-MANY

Four associative entities resolve many-to-many links

Role ⇄ Permission

↓

`ROLE_PERMISSION`

Composite key (role_id, permission_id)

User ⇄ Organization

↓

`ORGANIZATION_MEMBER`

Authorized representatives, with member_role, approved_at, is_active

Event ⇄ Category

↓

`EVENT_CATEGORY`

Item types a drive accepts

Event ⇄ Item

↓

`EVENT_ITEM`

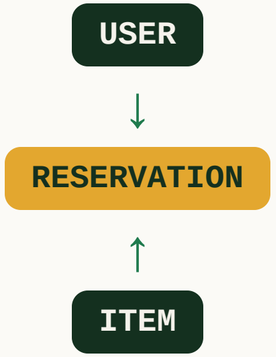
Contributed items, with contributor and distribution_status

USE CASE 1 · CAMPUS USER

Reserving an item

Functional requirement

“Reserve or claim the item if it is available. The item becomes unavailable to other users while the reservation is active.”



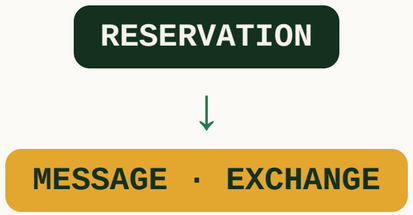
Field	Entity	Supports
`reserved_by_user_id`	RESERVATION	Who reserved it
`item_id`	RESERVATION	Which item
`receiver_organization_id`	RESERVATION	Optional: received for a department or org
`reserved_at, expires_at`	RESERVATION	Reservation date and auto-expiry
`status`	RESERVATION	Active, Completed, Cancelled, Expired
`status`	ITEM	Available → Reserved, hidden from others

USE CASE 2 · CAMPUS USER & DEPARTMENT REP

Arranging and completing an exchange

Functional requirement

“The giver and receiver can confirm that the exchange was completed, causing the item and reservation statuses to update.”



Field	Entity	Supports
`reservation_id`	MESSAGE	Chat to arrange the pickup
`reservation_id (UK)`	EXCHANGE	One exchange per reservation
`pickup_time, pickup_location`	EXCHANGE	When and where
`giver_confirmed_at`	EXCHANGE	Giver confirms handover
`receiver_confirmed_at`	EXCHANGE	Receiver confirms pickup
`completed_at`	EXCHANGE	Both confirmed → item and reservation Completed

USE CASE 3 · STUDENT ORGANIZATION REP

Running a collection drive

Functional requirement

“Create a giveaway or collection drive… title, description, date, location, accepted item types, participation instructions.”



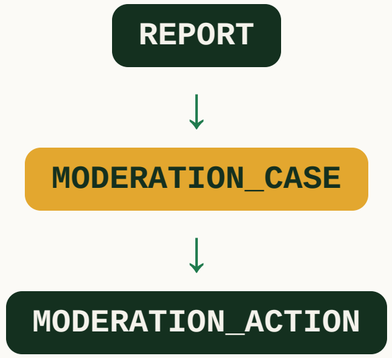
Field	Entity	Supports
`organization_id, created_by_user_id`	COLLECTION_EVENT	Hosting org and its rep
`title, location, start_at, end_at`	COLLECTION_EVENT	Event details and dates
`participation_instructions`	COLLECTION_EVENT	How to take part
`event_id, category_id`	EVENT_CATEGORY	Accepted item types
`contributor_user_id`	EVENT_ITEM	Who donated each item
`distribution_status`	EVENT_ITEM	Pending → Accepted → Distributed

USE CASE 4 · COMMUNITY MODERATOR

Reporting a problem and moderating it

Functional requirement

“Create and update moderation cases, record findings and actions, assign case status, and close resolved cases.”



Field	Entity	Supports
`reporter_user_id, reason`	REPORT	Who reported it and why
`item_id, exchange_id, event_id`	REPORT	What was reported (at least one)
`report_id (UK)`	MODERATION_CASE	One case per report
`findings, resolution, status`	MODERATION_CASE	Case record and outcome
`escalated_to_user_id`	MODERATION_CASE	Escalation to an Administrator
`action_type, restriction_ends_at`	MODERATION_ACTION	Warnings and temporary restrictions

DATA DICTIONARY

Every field documented: RESERVATION as an example

Field	Description	Type	Key	Required
reservation_id	Unique identifier for the reservation	INT	PK	Yes
item_id	Item being reserved	INT	FK	Yes
reserved_by_user_id	User who made the reservation	INT	FK	Yes
receiver_organization_id	Department or organization receiving the item	INT	FK	No
reserved_at	When the reservation was made	DATETIME	Neither	Yes
expires_at	When it expires if not completed	DATETIME	Neither	Yes
status	Active, Completed, Cancelled, Expired	VARCHAR(30)	Neither	Yes
pickup_note	Notes for arranging pickup	TEXT	Neither	No

127 fields in 20 entities · Excel and Markdown versions in GitHub: Data Dictionary folder

DATA DICTIONARY

Six fields worth a closer look

`ITEM.owner_organization_id`

INT · FK · optional

Empty: owned by a person. Filled: a department or student org listing.

`ITEM.status`

VARCHAR(30) · required

Available, Reserved, Completed, Cancelled or Removed. Drives search results.

`ITEM.estimated_value`

DECIMAL(10,2) · optional

Powers the “money saved” and reuse-value reports.

`RESERVATION.expires_at`

DATETIME · required

Set from an admin setting, so unclaimed items return to Available.

`EXCHANGE.reservation_id`

INT · FK · unique

The unique constraint makes Reservation → Exchange one-to-one.

`USER.password_hash`

VARCHAR(255) · required

Only a hash is stored, never the plain-text password.

BUSINESS RULES

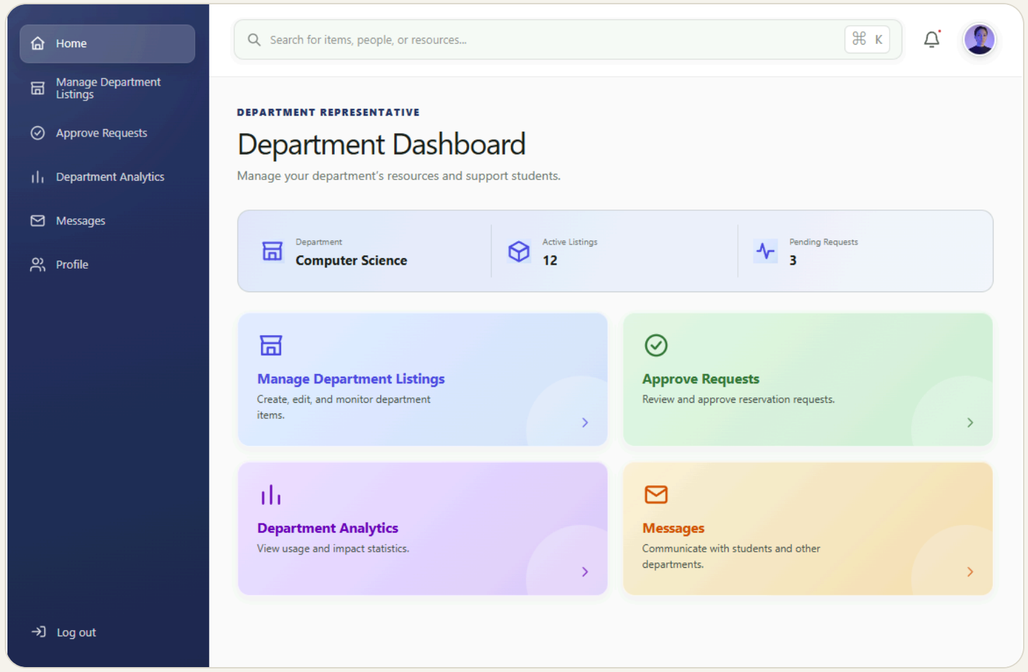
Rules the database design enforces

Rule	How the design enforces it
A user email must be unique	UNIQUE constraint on USER.email
An item can have only one active reservation at a time	Check ITEM.status and insert the reservation in one transaction with a row lock
Every item must belong to a category	ITEM.category_id is a required foreign key
A reservation expires after a set period (e.g. 24 hours) if not picked up	RESERVATION.expires_at from the admin's SYSTEM_SETTING
Exchanges need confirmation from both parties	Completed only when both giver_confirmed_at and receiver_confirmed_at are set
Representatives manage only their own organization's records	Active ORGANIZATION_MEMBER row required, plus ROLE_PERMISSION checks

Full list: Business Rules folder in GitHub

REVIEW OF SPRINT I UI DESIGNS

What the screens changed in the database



Sprint I: Department Representative dashboard

Added: MESSAGE

Every role's dashboard has a Messages page, so messages between users are now stored.

Added: ITEM.estimated_value

Department Analytics shows impact, which needs an item value to total.

Confirmed

Multiple photos (ITEM_IMAGE), notification bell (NOTIFICATION), dashboard counts derived from ITEM and COLLECTION_EVENT.

OPEN QUESTIONS

To resolve before implementation

1 · Approval states

“Approve Requests” and “Pending Listings” screens need a Pending status on RESERVATION and ITEM.

2 · List Item form

The form has no condition or pickup location, which the requirements and ERD need. Update the UI.

3 · Rules not yet modeled

Verified campus email, giveaway quantity, one reservation per category. Add fields or drop the rules?

4 · Moderation reasons

Rules say every action needs a reason; MODERATION_ACTION.notes is optional. Make it required?

5 · Admin audit log

Suspensions outside a moderation case need a logged justification. Add an audit-log table?

6 · Bulk requests

Student orgs can “request items for events”, but only giving is modeled. In scope?

GITHUB

All Sprint II materials are in the repository

```
Sprint Cycles/Sprint Cycle 2/
├─ ERD/
│  ├─ Sprint-II-ERD.pdf · .png
│  ├─ ERD-1 … ERD-4 (areas)
│  └─ README.md
├─ Data Dictionary/
│  ├─ Data-Dictionary.md
│  └─ …Data_Dictionary.xlsx
├─ Business Rules/
│  └─ Business Rules Sprint II.docx
└─ README.md
```

Member	Sprint II contribution
Naser Halloum	Entity-Relationship Diagram
Ondrea Wagner	Data dictionary, ERD revisions
Lloyd Lewis	Data dictionary, sprint docs
Isaiah Miller	Business rules, requirements review

Every member pushed Sprint II work and pulled the current repository.

github.com/LewisLloyd96/Rattler-Resource-Rescue-System

NEXT · SPRINT CYCLE III

From design to a working database

Resolve

Answer the six open questions and update the ERD.

Implement

Django models and migrations on MySQL, from the data dictionary.

Test

Turn the business rules into validation and pytest cases.

Questions?